

PROJECT F.A.R.C.

VOID EXTRACTION

SUBMISSION	Design Championship 2026	CATEGORY	Game Design (Senior)
TEAM ID	Sou_2_GaG_Senior_2026_26269	ENGINE	Godot 4.x (GDScript)
PROJECT STATUS	• COMPLETE (Windows .exe Executable)	PLANET CODE	ZEVA-Prime

1. EXECUTIVE SUMMARY

MISSION BRIEFING: *PROJECT F.A.R.C.: VOID EXTRACTION* is a high-tension, procedural 3D sci-fi survival and extraction protocol. The player, acting as a Forward Acquisition and Resource Command (F.A.R.C.) contractor, must navigate a hostile, dynamically generated alien environment on Planet Zeva.

The core objective is to recover critical drone parts scattered across a highly volatile sector, reactivate the Central Deacon beacon, survive an escalating onslaught of biomechanical native entities, and successfully extract via a dimensional portal before total system collapse.

2. UNIQUE SELLING POINTS (USPS)

ZERO-EXTERNAL-ASSET ARCHITECTURE

The entire visual landscape, including complex alien entities and structures, is built dynamically at runtime using pure GDScript, built-in Godot primitive meshes, and procedural `NoiseTexture2D` maps. No external `.obj`, `.glb`, or texture files are utilized, proving advanced engine mastery.

SMART PATHFINDING AI

Enemies utilize advanced `NavigationAgent3D` systems with dynamic Line-of-Sight (`RayCast3D`) interruption, allowing them to intelligently navigate procedural obstacle layouts and transition seamlessly between Patrol, Hunt, and Swarm states.

3. CHRONOLOGICAL MISSION ARCHITECTURE

The gameplay loop is governed by a strict Game Manager script that pushes the player through escalating phases of tension. The UI is diegetic, utilizing a typewriter-style communication panel to issue orders from F.A.R.C. Command.

PHASE 0: INFILTRATION & BRIEFING

The player drops onto Planet Zeva, temporarily locked in position. The communication panel boots up with a typewriter effect.

Commander: *"There is a bit of an issue... the parts I was sending you accidentally got scattered from the sky... Now you have to find all the parts and bring them to the central beacon!"*

Upon completion, the player's HUD (Health 100/100 and Objective Panel) initializes, and full movement is granted.

PHASE 1: SCAVENGE PROTOCOL

Objective: Collect 10 Scattered Drone Parts.

10 procedural components (Constructed via ``CylinderMesh`` + cyan glowing ``V`` ``BoxMesh`` emblems) are scattered across the map. The player must navigate the dark environment using limited visibility to locate all 10 items.

PHASE 2 & 3: THE DEACON ERROR & SYSTEM FAILURE

Upon delivering the parts to the Central Deacon [E], a critical error triggers. A massive burst of orange ``GPUParticles3D`` erupts, and the HUD flashes red.

Objective Update: Collect 5 Emergency Fuses.

5 highly emissive, pulsating yellow fuse nodes spawn. Simultaneously, the planetary defense sequence activates, initializing the Smart Alien wave spawners (Module 1). The player is now actively hunted while searching for fuses.

PHASE 4: OVERLOAD & THE SWARM

Inserting the fuses [P] into the Deacon triggers a 10-second imminent overload countdown. A global state override forces ALL active aliens into the ``ATTRACTED`` state. They abandon combat and sprint toward the Central Deacon, forming a massive swarm. The player must immediately sprint to the nearby **Metal Bunker** and lock themselves inside [E].

PHASE 5: DIMENSIONAL PORTAL EXTRACTION

0s-5s: Viewed safely through the bunker slit, a blinding orange sphere expands rapidly from the Deacon, enveloping the swarm.

5s-10s: The blast clears the map of all hostile entities (``queue_free()``).

10s+: The bunker opens. A vertical purple dimensional portal (`Emissive`SphereMesh``)

4. THREAT ASSESSMENT: SMART ALIEN AI

To adhere to the strict zero-external-asset mandate, the hostile native entities ("Aliens") are dynamically assembled at runtime via script using primitive geometries, procedural textures, and trigonometric animation sequences.

PROCEDURAL MESH ASSEMBLY

- **Armor:** Torso is a `CapsuleMesh`` with overlapping flat `BoxMesh`` nodes simulating rigid carapace plates.
- **Material:** A GDScript-generated `StandardMaterial3D`` utilizing `FastNoiseLite`` for a dark crimson, biomechanical scaly texture (Metallic: 0.7).
- **Appendages:** 4 angled `CylinderMesh`` back spikes with neon orange emissive tips (`Color(1, 0.5, 0)``).
- **Weapon:** Right arm merges primitives into a plasma rifle, housing a `Node3D`` muzzle and red `OmniLight3D``.

ALGORITHMIC LOCOMOTION

Instead of imported skeletal animations, the entity utilizes a 4-legged spider/mech leg assembly. Movement is calculated in the `_physics_process`` utilizing sine waves:

```
if velocity.length() > 0.1:
    Leg_FL.rot.x = sin(Time * 0.012) * 0.4
    Leg_FR.rot.x = -sin(Time * 0.012) * 0.4
```

This creates an unsettling, highly organic crawling motion that scales perfectly with the `NavigationAgent3D`` velocity.

AI STATE MACHINE ARCHITECTURE

STATE	LOGIC & BEHAVIOR
PATROL_SEARCH	Utilizes <code>NavigationServer3D`</code> to acquire random valid points on the generated NavMesh, intelligently pathing around scattered luggage, trees, and monoliths.
HUNT_PLAYER	Triggered by a <code>RayCast3D`</code> Line-of-Sight check. If the player intersects the ray, the Nav target dynamically updates to the player's real-time position.
SHOOTING	If distance < 10 units and LoS is clear, movement halts. The entity faces the player and instantiates a glowing red energy projectile (Velocity-based <code>RigidBody3D`</code>) every 1.2 seconds.
ATTRACTED	A global event override (Phase 4). Discards all combat logic and sprints directly toward the Central Deacon coordinates (<code>Vector3(0,0,0)`</code>).

DYNAMIC WAVE SPAWNER

The `AlienSpawner.gd`` controller maintains a constant ecosystem pressure. It tracks active entities; when the global count drops to 5 or fewer, it instantly instantiates a reinforcement wave of 5 new entities at randomized peripheral NavMesh points, ensuring the player is never truly safe.

5. ENVIRONMENTAL DESIGN & LIGHTING

Planet Zeva is designed to feel oppressive, expansive, and alien. The environment relies entirely on procedural scattering and advanced lighting techniques to create atmosphere, bypassing the need for heavy 3D asset files.

FIG 2.0 - THE CENTRAL DEACON (ELEVATION)



THE CENTRAL DEACON / BEACON

The primary landmark and narrative anchor of the extraction zone. Placed precisely at map origin ``Vector3(0,0,0)``. Constructed via script:

- **Core:** Towering dark metallic ``CylinderMesh``.
- **Energy Ring:** Massive cyan ``TorusMesh`` with an extreme emission energy multiplier (``energy = 5.0``).
- **Sky-Beam:** A vertical ``SpotLight3D`` firing straight up, acting as a visual guidepost for the player from any point on the map.

LIGHTING & POST-PROCESSING

The mood is established through a globally unified lighting setup:

- **Directional Light:** Set to moonlight blue (``Color(0.25, 0.38, 0.6)``) at an energy level of 0.45. Angle positioned at `(-45, -35, 0)` for long, dramatic shadows.
- **Ambient Light:** Deep navy (``Color(0.1, 0.15, 0.25)``).
- **WorldEnvironment:** Glow and Bloom settings are heavily pushed to ensure all procedural emissive meshes (Alien eyes, spikes, the Deacon, portal) create harsh, highly visible light-bleed in the dark.

PROCEDURAL TOPOGRAPHY DECOR

Empty spaces are procedurally populated via a scatter script. 12-15 glowing alienoid structures—built from jagged ``PrismMesh`` nodes simulating monoliths with purple/cyan veins, alongside clusters of glowing green and magenta ``SphereMesh`` pods—are distributed across the terrain. This breaks up sightlines and provides organic cover during firefights.

6. DEVELOPMENT & SUBMISSION SPECIFICATIONS

PROJECT F.A.R.C. was developed explicitly to highlight programming logic, procedural generation, and game-state management within the Godot Engine, purposefully avoiding reliance on pre-made external assets.

ZERO-EXTERNAL-ASSET DIRECTIVE

Technical Constraint Met: The entire project contains 0 external `.glb``, `.obj``, `.png``, or `.wav`` files. Every visual element, from the complex multi-limbed smart aliens to the glowing environmental monoliths, is generated using Godot's built-in primitive geometries, `StandardMaterial3D``, and noise-based textures generated entirely via GDScript logic.

PLAYER INPUT PROTOCOL

ACTION	INPUT MAPPING	ACTION	INPUT MAPPING
Movement (Omni)	[W] [A] [S] [D]	Sprint / Evade	[Left Shift]
Camera Look	Mouse Axis	Fire Weapon	[Left Mouse Button]
Equip Weapon	[E]	Interact (Deacon / Bunker)	[E]
Insert Emergency Fuses	[P]	Jump	[Spacebar]

EXPORT & DELIVERY INTEGRITY

To comply with competition accessibility rules while maintaining school privacy directives, the project has been packaged via Godot's internal exporter as a standalone Windows executable (`.exe`` with embedded `.pck``). The build strips debug nodes to ensure maximum performance and is delivered via a secure, unlisted cloud repository ensuring immediate access for the judging panel.

// END OF REPORT //

F.A.R.C. SYSTEM LOG OUT SUCCESSFUL